

P2374R3

views::cartesian_product

Published Proposal, 2021-12-7

This version:

<http://wg21.link/P2374>

Authors:

Sy Brand (sy.brand at microsoft dot com)

Michał Dominiak (griwes at griwes dot info)

Audience:

LEWG

Project:

ISO/IEC JTC1/SC22/WG21 14882: Programming Language — C++

Abstract

This paper proposes `std::ranges::cartesian_product_view` for taking the cartesian product of multiple forward ranges.

Table of Contents

1 Changelog

- 1.1 Changes from r2
- 1.2 Changes from r1
- 1.3 Changes from r0

2 Motivation

3 Design

- 3.1 Minimum Range Requirements
- 3.2 Tuple or pair
- 3.3 [reference_type](#)
- 3.4 Empty cartesian product view
- 3.5 Common Range
- 3.6 Bidirectional Range
- 3.7 Random Access Range
- 3.8 Initial Range Argument Special-Casing
- 3.9 Sized Range
- 3.10 Naming
- 3.11 Pipe Support
- 3.12 Borrowed Range?

4 Implementation

5 Wording

- 5.1 Addition to `<ranges>`
- 5.2 Range adaptor helpers [`range.adaptor.helpers`]
- 5.3 Cartesian product view [`range.cartesian`]
 - 5.3.1 Overview [`range.cartesian.overview`]
 - 5.3.2 Class template [cartesian_product_view](#) [`range.cartesian.view`]

- 5.3.3 Class template `cartesian_product_view::iterator` [ranges.cartesian.iterator]
- 5.4 Feature-test macro

6 Acknowledgements

§ 1. Changelog

§ 1.1. Changes from r2

- Make the size and difference types implementation-defined and only recommend requirements for them.

§ 1.2. Changes from r1

- Add wording adding `cartesian_product_view` into the synopsis of `<ranges>`
- Made the constructor of the view `explicit`
- Added a design section about not making `cartesian_product_view` a borrowed range
- Added a feature test macro
- Use the verbiage about the return type of `size` to also define the iterator's `difference_type`
- Relax the requirements on the first range, allowing it to be an input range
- Relax the requirements on the ranges for the view to be a common range - it now only depends on the properties of the first range
- Add `operator-` overloads that allow computing the distance between an iterator and a sentinel
- Various wording fixes

§ 1.3. Changes from r0

- Specify the return type of `size`
- Add design section on the first range argument

§ 2. Motivation

`Cartesian product` is a fundamental mathematical construct. There should be a `cartesian_product_view` which generates the cartesian product of any number of ranges, e.g.:

Before	After
<pre>std::vector<int> a,b,c; for (auto&& ea : a) { for (auto&& eb : b) { for (auto&& ec : c) { use(ea,eb,ec); } } }</pre>	<pre>std::vector<int> a,b,c; for (auto&& [ea,eb,ec] : std::views::cartesian_product(a,b,c)) { use(ea,eb,ec); }</pre>

This is especially useful for composing with other views, or dealing with parameter packs of ranges:

Before	
<pre>template <std::size_t N = 0, class F, class Res, class Tuple, class... Args> auto find_tuples_satisfying_impl(F f, Res& res, Tuple const& ranges, Args const&... args) requires(N == std::tuple_size_v<std::remove_cvref_t<Tuple>>) {</pre>	<pre>template <class F, std::ranges::for requires(std::regular_invocable<F, std::ranges::range_refere auto find_tuples_satisfying(F f, Vs return std::views::cartesian_prod</pre>

```

    if (std::invoke(f, args...)) {
        res.push_back(std::make_tuple(args...));
    }
}

template <std::size_t N = 0, class F,
          class Res, class Tuple, class... Args>
auto find_tuples_satisfying_impl(
    F f, Res& res, Tuple const& ranges, Args const&... args) {
    for (auto&& e : std::get<N>(ranges)) {
        find_tuples_satisfying_impl<N+1>(f, res, ranges, args..., e);
    }
}

template <class F, std::ranges::forward_range... Vs>
requires(std::regular_invocable<
    F, std::ranges::range_reference_t<Vs>...>)
auto find_tuples_satisfying(F f, Vs&&... vs) {
    std::vector<std::tuple<std::ranges::range_value_t<Vs>...>> res;
    find_tuples_satisfying_impl(
        f, res, std::tuple(std::forward<Vs&&>(vs)...));
    return res;
}

```

```

| std::views::filter([f](auto&&
| std::ranges::to<std::vector>(
}

```

§ 3. Design

§ 3.1. Minimum Range Requirements

This paper requires all ranges, except the first one, passed to [cartesian_product_view](#) to be forward ranges. See [Initial Range Argument Special-Casing](#) for discussion on the first argument being an input range.

§ 3.2. Tuple or pair

A potential design would be to use `std::tuple` as the `value_type` and `reference_type` of [cartesian_product_view](#). This paper uses `std::pair` if two ranges are passed and `std::tuple` otherwise. See [p2321](#) for motivation.

§ 3.3. reference_type

This paper uses `tuple-or-pair<ranges::reference_t<Vs>...>` as the reference type. See [p2214](#) for discussion of value types (particularly `pair` and `tuple`) as the `reference_type` for ranges, and [p2321](#) for wording of improvements for key use-cases.

§ 3.4. Empty cartesian product view

Trying to take the cartesian product view of 0 views will produce an `empty_view<tuple<>>`, in parity with Range-v3 and [p2321](#).

§ 3.5. Common Range

[cartesian_product_view](#) can be a common range if the first range either is common, or is sized and random access. This paper reflects this.

§ 3.6. Bidirectional Range

`cartesian_product_view` can be a bidirectional range if the underlying ranges, except for the first one, are bidirectional and common, or if they are random access and sized. Non-common bidirectional ranges are not supported because decrementing when one of the iterators is at the beginning of its range would require advancing the iterator to end, which may be linear in complexity.

We don't consider non-common, random access, sized ranges as worth supporting, so this paper requires bidirectional and common.

§ 3.7. Random Access Range

`cartesian_product_view` can be a random access range if all the underlying ranges are random access and sized, with the second requirement not applying to the first range. Sized ranges are required because when the view is incremented, the new states of the iterators are calculated modulo the size of their views.

We can't think of too many use-cases for this and it adds a fair bit of implementation burden, but this paper supports the necessary operations.

§ 3.8. Initial Range Argument Special-Casing

The first range passed to `cartesian_product_view` can be treated specially, since it is only passed through a single time. Therefore, the specification relaxes several constraints on the first range passed:

- It could be an input range instead of forward range
- It wouldn't need to be a sized range in order for the `cartesian_product_view` to be random-access or common
- It wouldn't need to be common in order for the `cartesian_product_view` to be bidirectional

Previous revisions of this paper didn't propose this feature, however the Ranges SG has requested that it be added to the paper.

§ 3.9. Sized Range

`cartesian_product_view` can be a sized range if all the underlying ranges are, in which case the size is the product of all underlying sizes. This is reflected in the paper.

§ 3.10. Naming

An alternative name is `std::ranges::product_view`. This paper uses `cartesian_product_view` as we believe it is more explicit in what its semantics are.

§ 3.11. Pipe Support

It may be possible to support syntax such as `vec1 | views::cartesian_product(vec2)` by either fixing the number of arguments allowed to the view, or adding a pipe operator to `cartesian_product_view` itself.

However, it's problematic for the same reason as `views::zip`, in that one cannot distinguish between the usages in `a | views::cartesian_product(b, c)` and `views::cartesian_product(b, c)` on its own. As such, this paper does not support this syntax.

§ 3.12. Borrowed Range?

It is possible to implement `cartesian_product_view` in such a way that it is a borrowed range if all the underlying ranges are borrowed. However, this requires that every iterator of the view stores two iterators per every range used for the cartesian product - one for the current position within that range, and another for the begin iterator, to allow wrapping

around back to the first element when the end of that range is reached. (For random access underlying ranges, it is conceivable to store an iterator and an offset, but this is not applicable in general.) If `cartesian_product_view` is not a borrowed view, the iterators only require storing one iterator per every range, and a pointer to the view itself.

Due to this, this paper does not make `cartesian_product_view` a borrowed range.

§ 4. Implementation

There are implementations of a cartesian product view in `Range-v3`, `cor3ntin::rangesnext`, and `tl::ranges`, among others.

§ 5. Wording

§ 5.1. Addition to `<ranges>`

Add the following to 24.2 [ranges.syn], Header `<ranges>` synopsis:

```
// ...
namespace std::ranges {
    // ...

    // [range.cartesian], cartesian product view
    template<input_range First, forward_range... Vs>
        requires (view<First> && (view<Vs> && ...))
    class cartesian_product_view;

    namespace views { inline constexpr unspecified cartesian_product = unspecified; }
}
```

§ 5.2. Range adaptor helpers [range.adaptor.helpers]

New section after Non-propagating cache [range.nonprop.cache]. Move the definitions of `tuple-or-pair`, `tuple-transform`, and `tuple-for-each` from Class template `zip_view` [range.zip.view] to this section:

```
namespace std::ranges {
    template <class... Ts>
        using tuple-or-pair = see-below; // exposition only

    template<class F, class Tuple>
    constexpr auto tuple-transform(F&& f, Tuple&& tuple) { // exposition only
        return apply([&]<class... Ts>(Ts&&... elements) {
            return tuple-or-pair<invoke_result_t<F&, Ts>...>(
                invoke(f, std::forward<Ts>(elements))...
            );
        }, std::forward<Tuple>(tuple));
    }

    template<class F, class Tuple>
    constexpr void tuple-for-each(F&& f, Tuple&& tuple) { // exposition only
        apply([&]<class... Ts>(Ts&&... elements) {
            (invoke(f, std::forward<Ts>(elements)), ...);
        }, std::forward<Tuple>(tuple));
    }
}
```

Given some pack of types Ts, the alias template `tuple-or-pair` is defined as follows:

1. If `sizeof...(Ts)` is 2, `tuple-or-pair<Ts...>` denotes `pair<Ts...>`.
2. Otherwise, `tuple-or-pair<Ts...>` denotes `tuple<Ts...>`.

§ 5.3. Cartesian product view [range.cartesian]

§ 5.3.1. Overview [range.cartesian.overview]

`cartesian_product_view` presents a `view` with a value type that represents the cartesian product of the adapted ranges.

The name `views::cartesian_product` denotes a customization point object. Given a pack of subexpressions `Es...`, the expression `views::cartesian_product(Es...)` is expression-equivalent to

- `*decay-copy*(views::empty<tuple<>>)` if `Es` is an empty pack,
- otherwise, `cartesian_product_view<views::all_t<decltype((Es))>...>(Es...)`.

[Example:

```
std::vector<int> v { 0, 1, 2 };
for (auto&& [a,b,c] : std::views::cartesian_product(v, v, v)) {
    std::cout << a << ' ' << b << ' ' << c << '\n';
    //0 0 0
    //0 0 1
    //0 0 2
    //0 1 0
    //0 1 1
    //...
}
```

-- end example]

§ 5.3.2. Class template `cartesian_product_view` [range.cartesian.view]

```
namespace std::ranges {
    template <class First, class... Vs>
    concept cartesian-product-is-random-access = // exposition only
        (random_access_range<First> && ... &&
         (random_access_range<Vs> && sized_range<Vs>));

    template <class R>
    concept cartesian-product-common-arg = // exposition only
        common_range<R> || (sized_range<R> && random_access_range<R>);

    template <class First, class... Vs>
    concept cartesian-product-is-bidirectional = // exposition only
        (bidirectional_range<First> && ... &&
         (bidirectional_range<Vs> && cartesian-product-common-arg<Vs>));

    template <class First, class... Vs>
    concept cartesian-product-is-common = // exposition only
        cartesian-product-common-arg<First>;

    template <class... Vs>
    concept cartesian-product-is-sized = // exposition only
        (sized_range<Vs> && ...);

    template<typename First, typename... Vs>
    concept cartesian-sentinel-is-sized = // exposition only
        sized_sentinel_for<sentinel_t<First>, iterator_t<First>> && ... && sized_range<Vs>;

    template <cartesian-product-common-arg R>
    auto cartesian-common-arg-end(R & r) {
        if constexpr (common_range<R>) {
            return std::ranges::end(r);
        }
        else {
            return std::ranges::begin(r) + std::ranges::size(r);
        }
    }
}
```

```

    }
}

template <input_range First, forward_range... Vs>
    requires (view<First> && ... && view<Vs>)
class cartesian_product_view
: public view_interface<cartesian_product_view<First, Vs...>> {
private:
    std::tuple<First, Vs...> bases_; // exposition only

    template<bool Const>
    struct iterator; // exposition only
public:
    constexpr cartesian_product_view() = default;
    constexpr explicit cartesian_product_view(First first_base, Vs... bases);

    constexpr iterator<false> begin()
        requires (!simple_view<First> || ... || !simple_view<Vs>);
    constexpr iterator<true> begin() const
        requires (range<const First> && ... && range<const Vs>);

    constexpr iterator<false> end()
        requires ((!simple_view<First> || ... || !simple_view<Vs>) &&
            cartesian-product-is-common<First, Vs...>);
    constexpr iterator<true> end() const
        requires (cartesian-product-is-common<const First, const Vs...>);
    constexpr default_sentinel_t end() const
        requires (!cartesian-product-is-common<const First, const Vs...>);

    constexpr *see below* size()
        requires cartesian-product-is-sized<First, Vs...>;
    constexpr *see below* size() const
        requires cartesian-product-is-sized<const First, const Vs...>;
};

template <class... Vs>
cartesian_product_view(Vs&&...)->cartesian_product_view<all_t<Vs>...>;

namespace views { inline constexpr unspecified cartesian_product = unspecified; }
}

```

```
constexpr explicit cartesian_product_view(First first_base, Vs... bases);
```

Effects: Initialises bases_ with std::move(first_base), std::move(bases)...

```
constexpr iterator<false> begin()
    requires (!simple_view<First> || ... || !simple_view<Vs>);
```

Effects: Equivalent to return iterator<false>(tuple-transform(ranges::begin, bases_));

```
constexpr iterator<true> begin()
    requires (range<First> && ... && range<Vs>);
```

Effects: Equivalent to return iterator<true>(tuple-transform(ranges::begin, bases_));

```
constexpr iterator<false> end()
    requires ((!simple_view<First> || ... || !simple_view<Vs>)
        && cartesian-product-is-common<First, Vs...>);
```

```
constexpr iterator<true> end() const
    requires(cartesian-product-is-common<const First, const Vs...>);
```

Effects: Let `is-const` be `true` for the `const`-qualified overload, and `false` otherwise. Equivalent to:

```
iterator<is-const> it(tuple-transform(ranges::begin, bases_));
std::get<0>(it.current_) = cartesian-common-arg-end(std::get<0>(bases_));
return it;
```

```
constexpr default_sentinel_t end() const
    requires (!cartesian-product-is-common<const First, const Vs...>) {
```

Effects: Equivalent to `return default_sentinel;`.

```
constexpr *see below* size()
    requires cartesian-product-is-sized<First, Vs...>;
constexpr *see below* size() const
    requires cartesian-product-is-sized<const First, const Vs...>;
```

The return type is an implementation-defined unsigned-integer-like type.

Recommended practice: The return type should be the smallest unsigned-integer-like type that is sufficiently wide to store the product of the maximum sizes of all underlying ranges. If there is no such type, the type is an implementation-defined unsigned-integer-like type.

Effects: Returns the product of the size of all ranges in `bases_`.

§ 5.3.3. Class template `cartesian_product_view::iterator` [ranges.cartesian.iterator]

```
namespace std::ranges {
template<input_range First, forward_range... Vs>
requires (view<First> && ... && view<Vs>))
template<bool Const>
class cartesian_product_view<First, Vs...>::iterator {
    maybe-const<Const, cartesian_product_view>* parent_; // exposition only
    tuple-or-pair<iterator_t<maybe-const<Const, First>>,
        iterator_t<maybe-const<Const, Vs>>...> current_{}; // exposition only

    template <size_t N = sizeof...(Vs)>
    void next(); // exposition only

    template <size_t N = sizeof...(Vs)>
    void prev(); // exposition only

    template <class Tuple>
    difference_type distance_to(Tuple t); // exposition only

public:
    using iterator_category = input_iterator_tag;
    using iterator_concept = *see below*;
    using value_type = tuple-or-pair<range_value_t<maybe-const<Const, First>>,
        range_value_t<maybe-const<Const, Vs>>...>;
    using reference = tuple-or-pair<reference_t<maybe-const<Const, First>>,
        reference_t<maybe-const<Const, Vs>>...>;
    using difference_type = *see below*;

    iterator() = default;
```



```

constexpr explicit iterator(tuple-or-pair<iterator_t<maybe-const<Const, First>>,
iterator_t<maybe-const<Const, Vs>>...> current);

constexpr iterator(iterator<!Const> i) requires Const &&
    (convertible_to<iterator_t<First>, iterator_t<maybe-const<Const, First>>> &&
    ... && convertible_to<iterator_t<Vs>, iterator_t<maybe-const<Const, Vs>>>);

constexpr auto operator*() const;
constexpr iterator& operator++();
constexpr iterator operator++(int);

constexpr iterator& operator--()
    requires (cartesian-product-is-bidirectional<maybe-const<Const, First>,
    maybe-const<Const, Vs>...>);
constexpr iterator operator--(int)
    requires (cartesian-product-is-bidirectional<maybe-const<Const, First>,
    maybe-const<Const, Vs>...>);

constexpr iterator& operator+=(difference_type x)
    requires (cartesian-product-is-random-access<maybe-const<Const, First>,
    maybe-const<Const, Vs>...>);
constexpr iterator& operator-=(difference_type x)
    requires (cartesian-product-is-random-access<maybe-const<Const, First>,
    maybe-const<Const, Vs>...>);

constexpr reference operator[](difference_type n) const
    requires (cartesian-product-is-random-access<maybe-const<Const, First>,
    maybe-const<Const, Vs>...>);

friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires (equality_comparable<iterator_t<maybe-const<Const, First>>> &&
    ... && equality_comparable<iterator_t<maybe-const<Const, Vs>>>);

friend constexpr bool operator==(const iterator& x, const std::default_sentinel_t&);

friend constexpr auto operator<(const iterator& x, const iterator& y)
    requires (random_access_range<maybe-const<Const, First>> &&
    ... && random_access_range<maybe-const<Const, Vs>>);
friend constexpr auto operator>(const iterator& x, const iterator& y)
    requires (random_access_range<maybe-const<Const, First>> &&
    ... && random_access_range<maybe-const<Const, Vs>>);
friend constexpr auto operator<=(const iterator& x, const iterator& y)
    requires (random_access_range<maybe-const<Const, First>> &&
    ... && random_access_range<maybe-const<Const, Vs>>);
friend constexpr auto operator>=(const iterator& x, const iterator& y)
    requires (random_access_range<maybe-const<Const, First>> &&
    ... && random_access_range<maybe-const<Const, Vs>>);

friend constexpr auto operator<==(const iterator& x, const iterator& y)
    requires ((random_access_range<maybe-const<Const, First>> &&
    ... && random_access_range<maybe-const<Const, Vs>>) &&
    (three_way_comparable<iterator_t<maybe-const<Const, First>>> &&
    ... && three_way_comparable<iterator_t<maybe-const<Const, Vs>>>));

friend constexpr iterator operator+(const iterator& x, difference_type y)
    requires (cartesian-product-is-random-access<maybe-const<Const, First>,
    maybe-const<Const, Vs>...>);
friend constexpr iterator operator+(difference_type x, const iterator& y)
    requires (cartesian-product-is-random-access<maybe-const<Const, First>,
    maybe-const<Const, Vs>...>);
friend constexpr iterator operator-(const iterator& x, difference_type y)
    requires (cartesian-product-is-random-access<maybe-const<Const, First>,
    maybe-const<Const, Vs>...>);
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
    requires (sized_sentinel_for<iterator_t<maybe-const<Const, First>>,

```

```

        maybe-const<Const, First>> &&
        ... && sized_sentinel_for<iterator_t<maybe-const<Const, Vs>>,
        maybe-const<Const, Vs>>);

friend constexpr operator-(iterator i, default_sentinel_t)
    requires cartesian-sentinel-is-sized<Vs...>;
friend constexpr operator-(default_sentinel_t, iterator i)
    requires cartesian-sentinel-is-sized<Vs...>;

friend constexpr auto iter_move(const iterator& i) noexcept(see below);

friend constexpr void iter_swap(const iterator& l, const iterator& r) noexcept(see below)
    requires (indirectly_swappable<iterator_t<maybe-const<Const, First>>> && ... &&
        indirectly_swappable<iterator_t<maybe-const<Const, Views>>>);
};
}

```

`iterator::iterator_concept` is defined as follows:

- If `cartesian-product-is-random-access<First, Vs...>` is modeled, then `iterator_concept` denotes `random_access_iterator_tag`.
- Otherwise, if `cartesian-product-is-bidirectional<First, Vs...>` is modeled, then `iterator_concept` denotes `bidirectional_iterator_tag`.
- Otherwise, if `First` models `forward_range`, then `iterator_concept` denotes `forward_iterator_tag`.
- Otherwise, `iterator_concept` denotes `input_iterator_tag`.

`iterator::difference_type` is an implementation-defined signed-integer-like type.

Recommended practice: `iterator::difference_type` should be the smallest signed-integer-like type that is sufficiently wide to store the product of the maximum sizes of all underlying ranges. If there is no such type, the type is an implementation-defined signed-integer-like type.

```

template <size_t N = sizeof...(Vs)>
void next() // exposition only

```

Effects: Equivalent to:

```

auto& it = get<N>(current_);
++it;
if constexpr (N > 0) {
    if (it == ranges::end(get<N>(parent_->bases_))) {
        it = ranges::begin(get<N>(parent_->bases_));
        next<N - 1>();
    }
}

```

```

template <size_t N = sizeof...(Vs)>
void prev() // exposition only

```

Effects: Equivalent to:

```
auto& it = std::get<N>(current_);
if (it == std::ranges::begin(std::get<N>(parent_>bases_))) {
    std::ranges::advance(it, cartesian-common-arg-end(std::get<N>(parent_>bases_)));
    if constexpr (N > 0) {
        prev<N - 1>();
    }
}
--it;
```

```
template <class Tuple>
difference_type distance_to(Tuple t); // exposition only
```

Effects: Let `scaled_size(N)` be `static_cast<difference_type>(std::ranges::size(get<N>(parent_>bases_)) * scaled_size(N+1) if $N < \text{sizeof} \dots (Vs)$, otherwise static_cast<difference_type>(1). Let scaled_distance(x, y, N) be static_cast<difference_type>(get<N>(current_) - get<N>(t)) * scaled_size(N+1). Returns the sum of scaled_distance(x, y, N) for every N in the interval [0, sizeof... (Vs)].`

```
constexpr explicit iterator(tuple-or-pair<iterator_t<maybe-const<Const, First>>,
    iterator_t<maybe-const<Const, Vs>>...> current);
```

Effects: Initializes `current_` with `std::move(current)`.

```
constexpr iterator(iterator<!Const> i) requires Const &&
    (convertible_to<iterator_t<First>, iterator_t<maybe-const<Const, First>>> &&
     ... && convertible_to<iterator_t<Vs>, iterator_t<maybe-const<Const, Vs>>>);
```

Effects: Initializes `current_` with `std::move(i.current_)`.

```
constexpr auto operator*() const;
```

Effects: Equivalent to:

```
return tuple-transform([](auto& i) -> decltype(auto) { return *i; }, current_);
```

```
constexpr iterator& operator++();
```

Effects: Equivalent to:

```
next();
return *this;
```

```
constexpr iterator operator++(int);
```

Effects: Equivalent to:

```
auto tmp = *this;
++*this;
return tmp;
```

```
constexpr iterator& operator--()
    requires (cartesian-product-is-bidirectional<maybe-const<Const, First>,
        maybe-const<Const, Vs>...>);
```

Effects: Equivalent to:

```
prev();
return *this;
```

```
constexpr iterator operator--(int)
    requires (cartesian-product-is-bidirectional<maybe-const<Const, First>,
        maybe-const<Const, Vs>...>);
```

Effects: Equivalent to:

```
auto tmp = *this;
--*this;
return tmp;
```

```
constexpr iterator& operator+=(difference_type x)
    requires (cartesian-product-is-random-access<maybe-const<Const, First>,
        maybe-const<Const, Vs>...>);
```

Effects: Sets the position of the iterators in `current_`:

- If $x > 0$, as if `next` was called x times.
- Otherwise, if $x < 0$, as if `prev` was called $-x$ times.
- Otherwise, no effect.

Complexity: Constant.

```
constexpr iterator& operator-=(difference_type x)
    requires (cartesian-product-is-random-access<maybe-const<Const, First>,
        maybe-const<Const, Vs>...>);
```

Effects: Equivalent to:

```
*this += -x;
return *this;
```

```
constexpr reference operator[](difference_type n) const
    requires (cartesian-product-is-random-access<maybe-const<Const, First>,
        maybe-const<Const, Vs>...>);
```

Effects: Equivalent to `return ((*this) + n);`.

```
friend constexpr bool operator==(const iterator& x, const iterator& y)
    requires (equality_comparable<iterator_t<maybe-const<Const, First>>> &&
        ... && equality_comparable<iterator_t<maybe-const<Const, Vs>>>);
```

Effects: Equivalent to `return x.current_ == y.current_;`.

```
friend constexpr bool operator==(const iterator& x, const std::default_sentinel_t&);
```

Effects: Equivalent to `return std::get<0>(x.current_) == std::ranges::end(std::get<0>(x.parent_>bases_));`.

```
friend constexpr auto operator<(const iterator& x, const iterator& y)
    requires (random_access_range<maybe-const<Const, First>> &&
        ... && random_access_range<maybe-const<Const, Vs>>);
```

Effects: Equivalent to `return x.current_ < y.current_;`.

```
friend constexpr auto operator>(const iterator& x, const iterator& y)
    requires (random_access_range<maybe-const<Const, First>> &&
        ... && random_access_range<maybe-const<Const, Vs>>);
```

Effects: Equivalent to `return y < x;`.

```
friend constexpr auto operator<=(const iterator& x, const iterator& y)
    requires (random_access_range<maybe-const<Const, First>> &&
        ... && random_access_range<maybe-const<Const, Vs>>);
```

Effects: Equivalent to `return !(y < x);`.

```
friend constexpr auto operator>=(const iterator& x, const iterator& y)
    requires (random_access_range<maybe-const<Const, First>> &&
        ... && random_access_range<maybe-const<Const, Vs>>);
```

Effects: Equivalent to `return !(x < y);`.

```
friend constexpr auto operator<=>(const iterator& x, const iterator& y)
    requires ((random_access_range<maybe-const<Const, First>> &&
        ... && random_access_range<maybe-const<Const, Vs>>) &&
        (three_way_comparable<maybe-const<Const, First>> &&
        ... && three_way_comparable<maybe-const<Const, Vs>>));
```

Effects: Equivalent to `return x.current_ <=> y.current_;`.

```
friend constexpr iterator operator+(const iterator& x, difference_type y)
    requires (cartesian-product-is-random-access<maybe-const<Const, First>,
        maybe-const<Const, Vs>...>);
```

Effects: Equivalent to `return iterator{ x } += y;`

```
friend constexpr iterator operator+(difference_type x, const iterator& y)
requires (cartesian-product-is-random-access<maybe-const<Const, First>,
        maybe-const<Const, Vs>...>);
```

Effects: Equivalent to `return y + x;`

```
friend constexpr iterator operator-(const iterator& x, difference_type y)
requires (cartesian-product-is-random-access<maybe-const<Const, First>,
        maybe-const<Const, Vs>...>);
```

Effects: Equivalent to `return iterator{ x } -= y;`

```
friend constexpr difference_type operator-(const iterator& x, const iterator& y)
requires (sized_sentinel_for<iterator_t<maybe-const<Const, First>>,
        maybe-const<Const, First>> &&
        ... && sized_sentinel_for<iterator_t<maybe-const<Const, Vs>>,
        maybe-const<Const, Vs>>);
```

Effects: Equivalent to `return x.distance_to(y.current_);`

```
friend constexpr operator-(iterator i, default_sentinel_t)
requires cartesian-sentinel-is-sized<Vs...>;
```

Effects: Let end-tuple be an object of a type that is a specialization of `std::tuple`, such that:

- `std::get<0>(end-tuple)` has the same value as `std::ranges::end(std::get<0>(i.parent_>bases_))`;
- `std::get<N>(end-tuple)` has the same value as `std::ranges::begin(std::get<N>(i.parent_>bases_))` for every N in the interval `[1, sizeof...(Vs) + 1)`.

Equivalent to: `return i.distance_to(end-tuple);`

```
friend constexpr operator-(default_sentinel_t s, iterator i)
requires cartesian-sentinel-is-sized<Vs...>;
```

Effects: Equivalent to `return -(i - s);`

```
friend constexpr auto iter_move(const iterator& i) noexcept(see below);
```

Effects: Equivalent to: `return tuple-transform(ranges::iter_move, i.current_);`

Remarks: The exception specification is equivalent to:

```
(noexcept(ranges::iter_move(declval<const iterator_t<maybe-const<Const, First>>&>())&&
noexcept(ranges::iter_move(declval<const iterator_t<maybe-const<Const, Views>>&>())&&
(is_nothrow_move_constructible_v<range_rvalue_reference_t<maybe-const<Const, First>>>&&
is_nothrow_move_constructible_v<range_rvalue_reference_t<maybe-const<Const, Views>>>&&))
```

```
friend constexpr void iter_swap(const iterator& l, const iterator& r) noexcept(see below)
requires (indirectly_swappable<iterator_t<maybe-const<Const, First>>>&& ... &&
indirectly_swappable<iterator_t<maybe-const<Const, Views>>>&&);
```

Effects: For every integer $0 \leq i < \text{sizeof} \dots (\text{Views}) + 1$, performs: `ranges::iter_swap(std::get<i>(l.current_), std::get<i>(r.current_))`.

Remarks: The exception specification is equivalent to the logical AND of the following expressions:
`noexcept(ranges::iter_swap(std::get<i>(l.current_), std::get<i>(r.current_)))` for every
integer $0 \leq i < \text{sizeof} \dots (\text{Views}) + 1$.

§ 5.4. Feature-test macro

Add the following macro definition to 17.3.2 [version.syn], Header `<version>` synopsis, with the value selected by the editor to reflect the date of adoption of this paper:

```
#define __cpp_lib_ranges_cartesian_product 20XXXXL // also in <ranges>
```

§ 6. Acknowledgements

Thank you to Christopher Di Bella, Corentin Jabot, Tim Song, and Barry Revzin for feedback and guidance.

Thank you to Tomasz Kamiński for help with getting the wording into a good shape.